

Matematično-fizikalni praktikum: Anharmonski oscilator

Žan Ambrožič (28231062)

27. oktober 2025

1 Naloga

Z diagonalizacijo poišči nekaj najnižjih lastnih vrednosti in lastnih valovnih funkcij za moteno Hamiltonko $H = H_0 + \lambda q^4$ ob vrednostih parametra $0 \leq \lambda \leq 1$.

2 Uvod

Enodimenzionalni linearni harmonski oscilator v paraboličnem potencialu opišemo z brezdimenzijsko Hamiltonovo funkcijo:

$$H_0 = \frac{p^2 + q^2}{2}; \quad T(p) = \frac{p^2}{2m}, \quad V(q) = \frac{m\omega^2 q^2}{2}. \quad (1)$$

Energijo lahko merimo v enotah $\hbar\omega$, gibalno količino v $\sqrt{\hbar m\omega}$ in dolžino v $\sqrt{\hbar/(m\omega)}$. Lastna stanja H_0 so valovne funkcije:

$$|n\rangle = \frac{e^{-q^2/2} \mathcal{H}_n(q)}{\sqrt{2^n n! \sqrt{\pi}}}, \quad (2)$$

kjer so $\mathcal{H}_n$ Hermiteovi polinomi. Lastne funkcije zadoščajo stacionarni Schrödingerjevi enačbi za lastne energije $E_n^0 = n + 1/2$:

$$H_0 |n^0\rangle = E_n^0 |n^0\rangle. \quad (3)$$

Matrika $\langle i|H_0|j\rangle$ je diagonalna z vrednostmi $\delta_{ij}(i + 1/2)$. Če dodamo anharmonski člen λq^4 , dobimo anharmonski oscilator:

$$H = H_0 + \lambda q^4. \quad (4)$$

Iščemo matrične elemente $\langle i|H|j\rangle$ v bazi lastnih stanj nemotenega harmonskega oscilatorja. Pričakovane vrednosti prehodnega matričnega elementa so:

$$q_{ij} = \langle i|q|j\rangle = \frac{\sqrt{i+j+1}\delta_{|i-j|,1}}{2} \quad (5)$$

in predstavljajo izbirno pravilo za električni dipolni prehod med nivoji harmonskega oscilatorja. V teoriji so matrike neskončne, v praksi pa se moramo omejiti na neko končno razsežnost (N), pri kateri previsoke člene izpustimo, kar se pozna pri natančnosti določitve najvišjih lastnih stanj.

3 Reševanje

3.1 Generiranje matrik q^4

Preden se lotimo izračunov lastnih vrednosti, moramo izračunati anharmonske popravke Hamiltonijana harmonskega oscilatorja. To lahko storimo na tri načine: izračunamo q in izvedemo dve matrični množenji do q^4 , direktno izračunamo elemente q^2 ter z enim matričnim množenjem pridemo do q^4 , ali pa direktno izračunamo elemente q^4 . Ker bomo imeli opravka tudi z velikimi matrikami, mora biti postopek implementiran učinkovito, zanimalo pa nas bo tudi, katera od metod je najprimernejša za dan N . Po formuli (5) lahko zapišemo pasovno matriko (v brezdimenzijski

obliki):

$$q = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 1 & 0 & \sqrt{2} & \ddots & \vdots \\ 0 & \sqrt{2} & 0 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \sqrt{n} \\ 0 & \cdots & 0 & \sqrt{n} & 0 \end{bmatrix} \quad (6)$$

Z množenjem zgornje matrike same s sabo opazimo vzorec, ki se z večanjem matrike ne spreminja (z izjemo zadnjega diagonalnega elementa, ki se popravi pri večji matriki). Diagonalni elementi sledijo pravilu $d_0(i) = 2i + 1$ (i je indeks vrstice/stolpca, indekse štejemo od 0), elementi drugih obdiagonal pa sledijo pravilu $d_2(i) = \sqrt{(i-1)(i)}$, kjer za i uporabimo večjega od indeksov vrstice/stolpca (formula je sicer razvidna tudi iz enačbe, podane v navodilih).

$$q^2 = \frac{1}{2} \begin{bmatrix} 1 & 0 & \sqrt{2} & 0 & \cdots & 0 \\ 0 & 3 & 0 & \sqrt{6} & \ddots & \vdots \\ \sqrt{2} & 0 & 5 & 0 & \ddots & 0 \\ 0 & \sqrt{6} & 0 & 7 & \ddots & d_2(n) \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & d_2(n) & 0 & d_0(n) \end{bmatrix} \quad (7)$$

Za q^4 ne želimo računati fakultet, saj so računsko bolj zamudnejše in ker bi vedno računali kvocient dveh fakultet, ki se razlikujeta za 2 ali 4. Za diagonalne člene sedaj dobimo $d'_0(i) = 3(2i^2 + 2i + 1)$, za drugi obdiagonalni (z enakim pravilom večjega indeksa) dobimo $d'_2(i) = 2(2i-2)\sqrt{i(i-1)}$, za četrti obdiagonalni pa $d'_4(i) = \sqrt{i(i-1)(i-2)(i-3)}$.

$$q^4 = \frac{1}{4} \begin{bmatrix} 3 & 0 & 6\sqrt{2} & 0 & 2\sqrt{6} & 0 & \cdots & 0 \\ 0 & 15 & 0 & 10\sqrt{6} & 0 & 2\sqrt{30} & \ddots & \vdots \\ 6\sqrt{2} & 0 & 39 & 0 & 28\sqrt{3} & 0 & \ddots & 0 \\ 0 & 10\sqrt{6} & 0 & 75 & 0 & 36\sqrt{5} & \ddots & d'_4(n) \\ 2\sqrt{6} & 0 & 28\sqrt{3} & 0 & 123 & 0 & \ddots & 0 \\ 0 & 2\sqrt{30} & 0 & 36\sqrt{5} & 0 & 183 & \ddots & d'_2(n) \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & d'_4(n) & 0 & d'_2(n) & 0 & d'_0(n) \end{bmatrix} \quad (8)$$

Numerično najnatančnejša je gotovo zadnja metoda, saj vključuje najmanj operacij na element, medtem ko pri prvi metodi prvotno izračunamo manj elementov in nato dvakrat izvedemo matrično množenje. Splošno matrično množenje zahteva $\mathcal{O}(n^3)$ oz. $\mathcal{O}(n^{2.8})$ operacij (Strassen), medtem ko je direkten izračun elementov sorazmeren z njihovim številom ($\mathcal{O}(n)$).

3.2 Numerični algoritmi

Poleg moje implementacije Householderjeve metode za tridiagonalizacijo simetrične matrike in diagonalizacijo simetrične tridiagonalne z Wilkinsonvimi premiki in Givensovimi rotacijami bomo preizkusili še vgrajene metode `numpy` in `scipy`, ki so navedene v tabeli 1.

Tabela 1: Pregled vgrajenih za diagonalizacijo simetričnih matrik

Funkcija	Tip matrike	Časovna zahtevnost
<code>numpy.linalg.eig(A)</code>	Splošna gosta matrika	$\mathcal{O}(n^3)$
<code>numpy.linalg.eigh(A)</code>	Gosta simetrična	$\mathcal{O}(n^3)$
<code>scipy.linalg.eig(A)</code>	Splošna gosta matrika	$\mathcal{O}(n^3)$
<code>scipy.linalg.eigh(A)</code>	Gosta simetrična	$\mathcal{O}(n^3)$
<code>scipy.linalg.eig_banded(bands)</code>	Simetrična pasovna matrika	$\mathcal{O}(bn^2)$
<code>scipy.linalg.eigh_tridiagonal(d, e)</code>	Simetrična tridiagonalna matrika	$\mathcal{O}(n^2)$

V paketu `scipy` so sicer na voljo še druge nižjenivojske LAPACK funkcije, vendar bi morale navedene višjenivojske zadostovati za primerjavo. Za tridiagonalizacijo simetrične matrike lahko uporabimo `scipy.linalg.hessenberg()`, saj je Hessenbergova forma za simetrične matrike ravno tridiagonalna. Treba je omeniti, da je bil uporabljen `numpy` 3.2.4, ki že privzeto uporablja paralelizacijo in za kakšne operacije tudi t.i. GPU-acceleration.

3.3 Potencial z dvema minimumoma

Na koncu bi radi raziskali še potencial z dvema minimumoma, katerega Hamiltonijan je:

$$H = \frac{p^2}{2} - 2q^2 + \frac{q^4}{10} = H_0 - \frac{3q^2}{2} + \frac{q^4}{10}, \quad (9)$$

delec je v potencialu (brezdimenzijska oblika):

$$V(q) = -2q^2 + \frac{q^4}{10}. \quad (10)$$

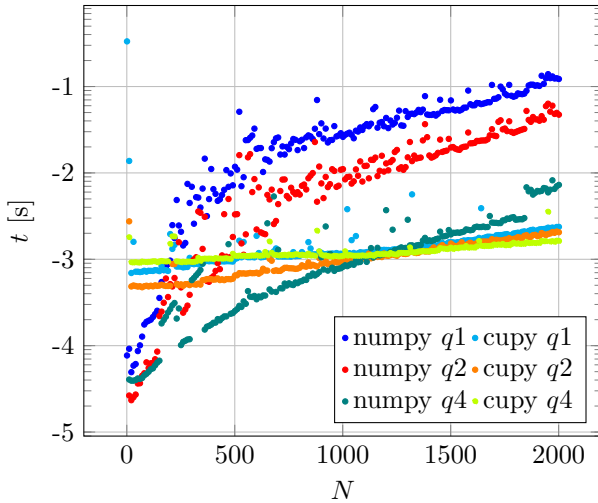
4 Rezultati

Vse simulacije so tekle na računalniku s procesorjem AMD Ryzen 7 5800x, 32 GB delovnega spomina (katerega kapaciteta ni bila omejevalni faktor) in grafično kartico GeForce RTX3060 z 12 GB spomina. Za operacije na grafični kartici je bila uporabljena knjižnica `cupy`, za procesorske pa `numpy` in `scipy`.

4.1 Generiranje matrik q^4

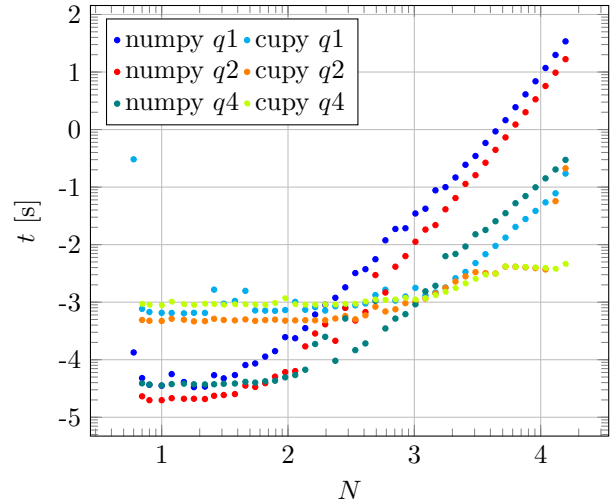
Pri generiranju z GPU smo med vsakim izračunom matrike poskrbeli za čiščenje spomina, da stari spomin ni mogel vplivati na izračun naslednje matrike. Zgornja meja velikosti matrik je bila omejena s spominom grafične kartice. Velikost matrike za graf 2 povečevali eksponentno (zaokroženo $1,2^i$), dosegli smo $i = 53$. Na procesorju bi lahko velikosti še povečevali, vendar je hitro rasel tudi čas izračuna.

Časovna zahtevnost različnih metod
v odvisnosti od N



Graf 1: Čas izračuna po različnih metodah (CPU, GPU).

Časovna zahtevnost različnih metod
v odvisnosti od N



Graf 2: Čas izračuna po različnih metodah (CPU, GPU) za zelo velike matrike.

Pomembno je še, koliko posamezna metoda porabi za kateri korak, kar je prikazano v 2. Opazimo, da procesor sploh pri večjih matrikah večji del časa porabi za matrično množenje, medtem ko le to teče (pričakovano) veliko hitreje na grafični kartici. Pri dvojnem matričnem množenju opazimo, da operacija ni tako učinkovita, kar bi verjetno lahko pripisali vmesnemu pisanju/branju spomina. Delež se ne spremeni znatno ne glede na to, ali uporabljamo $(q^2)^2$ ali direktno q^4 (v funkciji `linalg.matrix_power()`). Če bi se podatki med CPU in GPU prenašali v nezanemarljivem času, bi morda bilo smiselno narediti kombinirano metodo, kjer za določene velikosti matrik naredimo osnovo q^2 na

Tabela 2: Časovne zahtevnosti korakov pri izračunih matrik po različnih postopkih (CPU, GPU).

	N	Osnova	Generiranje	Množenje	t
CPU	100	q^1	0,63	0,37	$3,9 \cdot 10^{-4}$ s
		q^2	0,59	0,41	$6,9 \cdot 10^{-5}$ s
		q^4	1,00	–	$6,4 \cdot 10^{-5}$ s
	1000	q^1	0,27	0,73	$2,5 \cdot 10^{-2}$ s
		q^2	0,11	0,89	$9,2 \cdot 10^{-3}$ s
		q^4	1,00	–	$3,5 \cdot 10^{-3}$ s
	10000	q^1	0,05	0,95	9,1 s
		q^2	0,03	0,97	4,4 s
		q^4	1,00	–	$1,2 \cdot 10^{-1}$ s
GPU	100	q^1	0,17	0,83	$3,3 \cdot 10^{-2}$ s
		q^2	0,92	0,08	$1,1 \cdot 10^{-3}$ s
		q^4	1,00	–	$9,4 \cdot 10^{-4}$ s
	1000	q^1	0,19	0,81	$3,4 \cdot 10^{-2}$ s
		q^2	0,94	0,07	$1,6 \cdot 10^{-3}$ s
		q^4	1,00	–	$1,1 \cdot 10^{-3}$ s
	10000	q^1	0,63	0,37	$7,8 \cdot 10^{-2}$ s
		q^2	0,98	0,02	$7,0 \cdot 10^{-3}$ s
		q^4	1,00	–	$4,3 \cdot 10^{-3}$ s

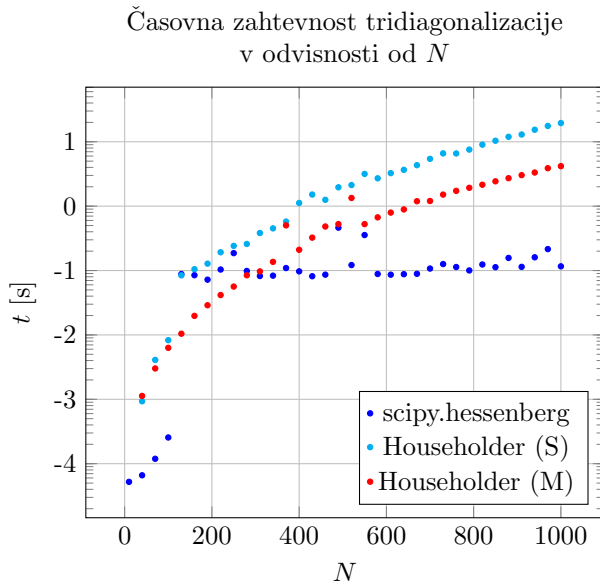
CPU, množimo pa na GPU. Je pa natančnost direktnega generiranja q^4 večja, saj vsak končni člen med generiranjem "preživi" manj operacij (ker je tudi začetnih členov več) – pravzaprav gre v tem primeru le za množenje celih števil (ki je točno) in šele nato en kvadratni koren, medtem ko pri postopkih z matričnim množenjem izračunamo korene že v osnovni matriki, s čimer vpeljemo končno natančnost vhodnih podatkov za matrično množenje, pri katere se numerične napake le še propagirajo. Še ena možna metoda bi bila, da bi začetne matrike generirali kot celoštevilске in šele na koncu korenili vse vrednosti, s čimer bi dosegli tako majhno numerično napako, kot jo ima direktno generiranje, pri generiranju osnove pa bi porabili malo manj časa (ni korenjenja, ni treba zgenerirati 3 različnih diagonal in jih zapisati na 5 različnih pasov v matriki), vendar bi se potem v metodi pojavilo še eno ozko grlo – na koncu bi morali izračunati korene primerne stopnje za vse elemente na petih pasovih, kar pa bi po moji oceni vzelo dlje časa, kot bi ga prihranili v postopku do te točke.

4.2 Diagonalizacija

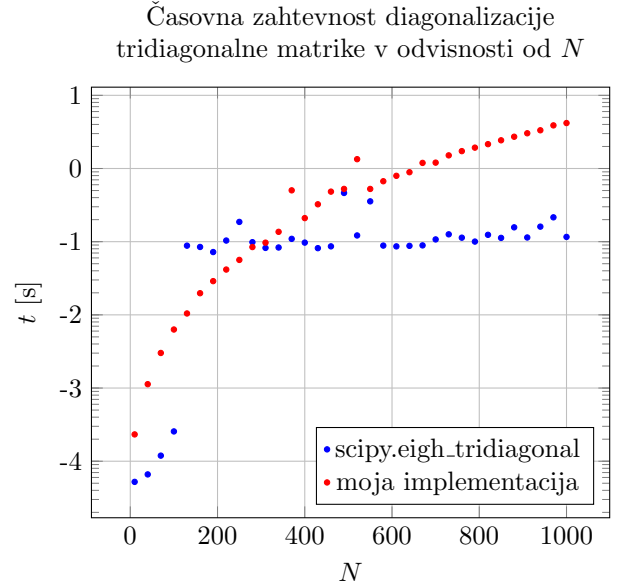
Za vse časovne teste nastavimo $\lambda = 0,5$. Matrike q^4 generiramo direktno, saj je to najnatančnejši in najhitrejši način.

4.2.1 Tridiagonalizacija

Primerjamo tri algoritme: `scipy.linalg.hessenberg()`, mojo implementacijo Householderjevega algoritma in implementacijo s spletne učilnice (povečamo največje dovoljeno število rekurzivnih klicev, da sprejme globino 1000). Rezultati so prikazani na grafu 3. Pričakovano je najhitrejša vgrajena metoda, vendar se tudi moja obnese kar v redu.



Graf 3: Časovna zahtevnost metod tridiagonalizacije.



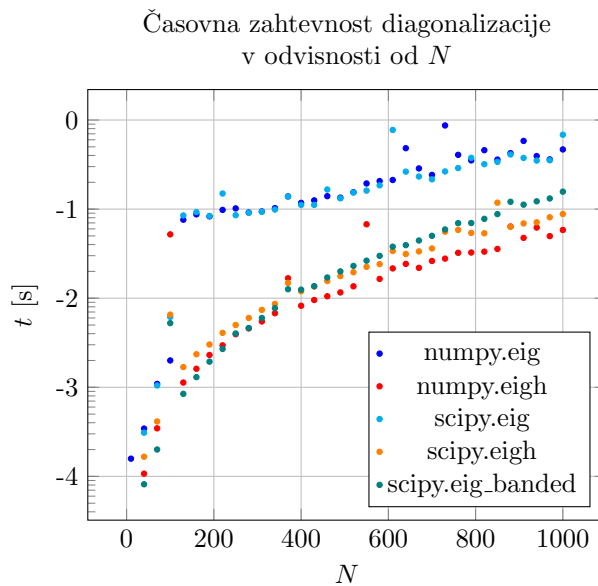
Graf 4: Časovna zahtevnost metod diagonalizacije tridiagonalne matrike.

4.2.2 Diagonalizacija tridiagonalne matrike

Za začetno tridiagonalizacijo uporabimo vgrajeno metodo `scipy.linalg.hessenberg()`. Za dokončno diagonalizacijo tridiagonalne matrike lahko uporabimo vgrajeno `scipy.linalg.eigh_tridiagonal()` in mojo implementacijo. Rezultati so prikazani na grafu 4, pričakovano je bila vgrajena metoda zopet hitrejša.

4.2.3 Vgrajena diagonalizacija

Vgrajene metode lahko diagonalizacijo izvedejo precej hitreje (graf 5) od moje implementacije (čas posamezne stopnje je daljši od časa celotne vgrajene diagonalizacije, zato nima smisla risati). Izkaže se, da je za velike matrike najhitrejša `numpy.eigh()`, smiselna kandidata bi bila še `scipy.eigh()` in `scipy.eig_banded()`, saj so ti algoritmi optimizirani za simetrične (oz. simetrične pasovne) matrike.

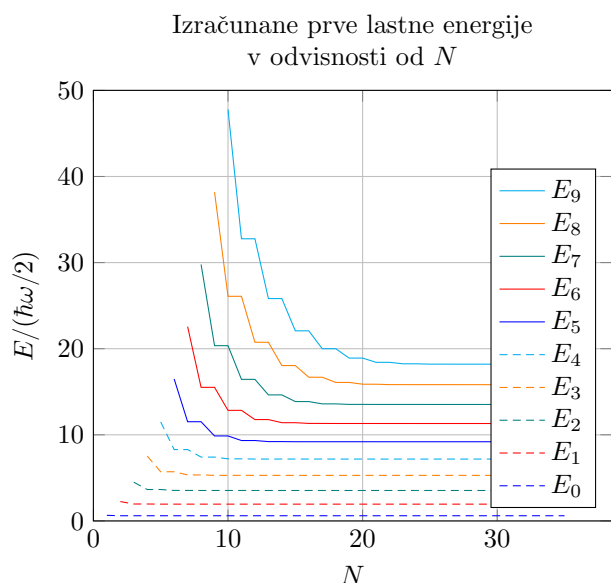


Graf 5: Časovna zahtevnost metod diagonalizacije za vgrajene metode.

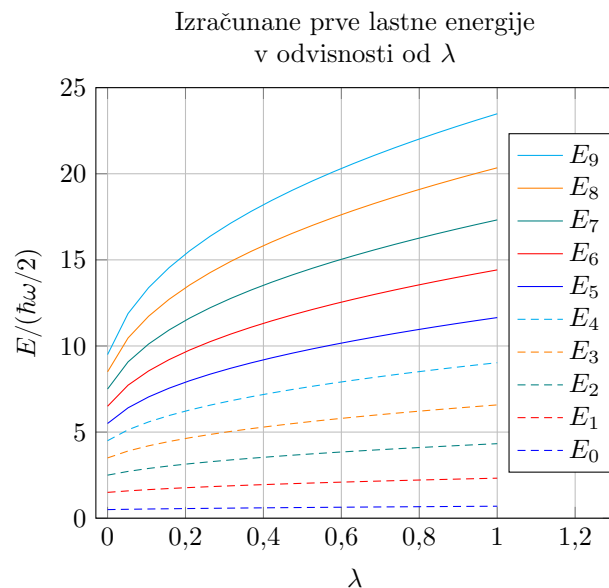
4.3 Lastna stanja

Sedaj se bomo osredotočili na lastne energije in lastne vektorje anharmonskega oscilatorja. Vse izračune bomo naredili z vgrajeno metodo `numpy.eigh()`. Zanimali nas bosta odvisnosti od $\lambda \in [0, 1]$ ter od velikosti Hamiltonijana.

4.3.1 Lastne energije



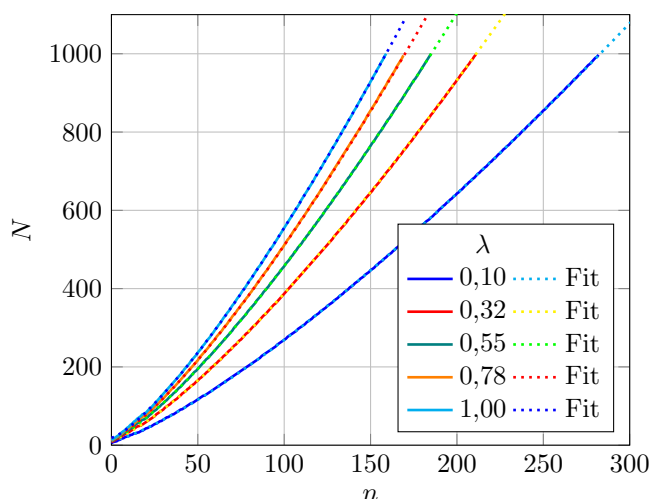
Graf 6: Prvih nekaj lastnih energij, izračunanih iz različno velikih Hamiltonijanov pri $\lambda = 0,4$.



Graf 7: Prvih nekaj lastnih energij v odvisnosti od anharmoničnosti (λ), $N = 1000$.

Kakor vidimo na grafu 6, višjih energij ne moremo dobro izračunati s premajhnimi Hamiltonijani (za $n > N$ sploh ne moremo dobiti vrednosti, za $n \lesssim N$ pa so lastne vrednosti previsoke). Koristno bi bilo vedeti, kako velik Hamiltonijan moramo vzeti, da dobimo želeno natančnost a so izračuni kar se da učinkoviti (tj. najmanjša zadostno natančna matrika).

Potrebna velikost matrike (N) za relativno natančnost 10^{-6} n -te lastne energije



Graf 8: Potrebna velikost Hamiltonijana za ujemanje znotraj napake 10^{-6} z referenčnim Hamiltonijanom velikosti $N = 1000$ za posamezno energijo.

Parabolični fiti ne ustrezajo tako dobro, poskusimo potenčnega:

$$N(n) = a + bn^c \quad (11)$$

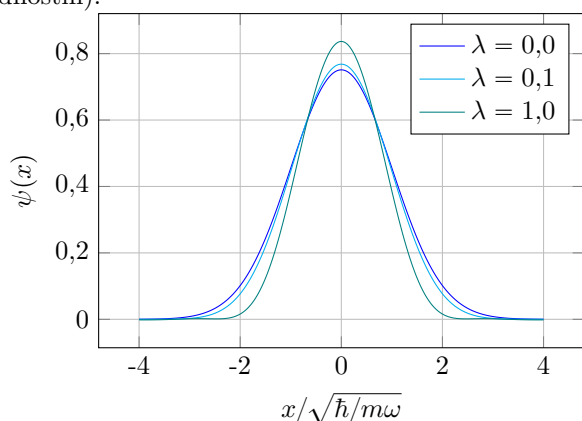
Tabela 3: Zaokroženi fit parametri potrebne velikosti matrike N za relativno natančnost 10^{-6} n -te lastne energije.

λ	a	b	c
0,10	$10,32 \pm 0,17$	$0,6795 \pm 0,0026$	$1,2903 \pm 0,0007$
0,33	$13,18 \pm 0,19$	$0,9535 \pm 0,0037$	$1,2968 \pm 0,0007$
0,55	$15,34 \pm 0,22$	$1,1142 \pm 0,0050$	$1,2994 \pm 0,0008$
0,78	$16,99 \pm 0,26$	$1,2396 \pm 0,0066$	$1,3004 \pm 0,0010$
1,00	$18,66 \pm 0,28$	$1,3391 \pm 0,0075$	$1,3015 \pm 0,0011$

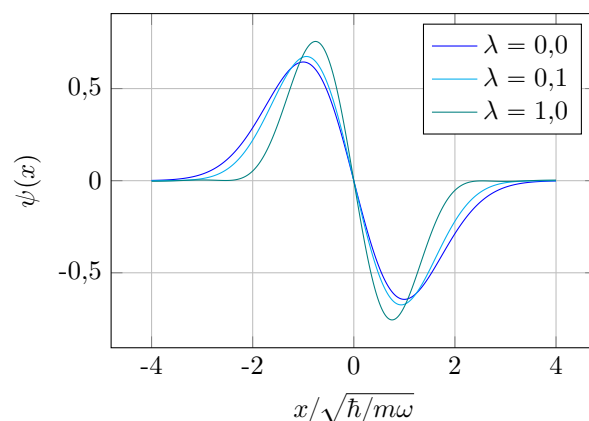
Izkaže se, da se funkcija lepo prilega na celotnem preizkusnem območju, potence pa so odvisne od anharmoničnosti (λ), vendar za naše območje $\lambda \leq 1$ dosežejo največ okoli 1,3, medtem ko je najmanjša potenca 1 pri $\lambda = 0$, kjer je Hamiltonijan harmonskega oscilatorja že diagonalen in so vse razpoložljive lastne vrednosti točne (pri Hamiltonijanu velikosti N dobimo $n = N$ točnih energijskih nivojev).

4.3.2 Lastni vektorji

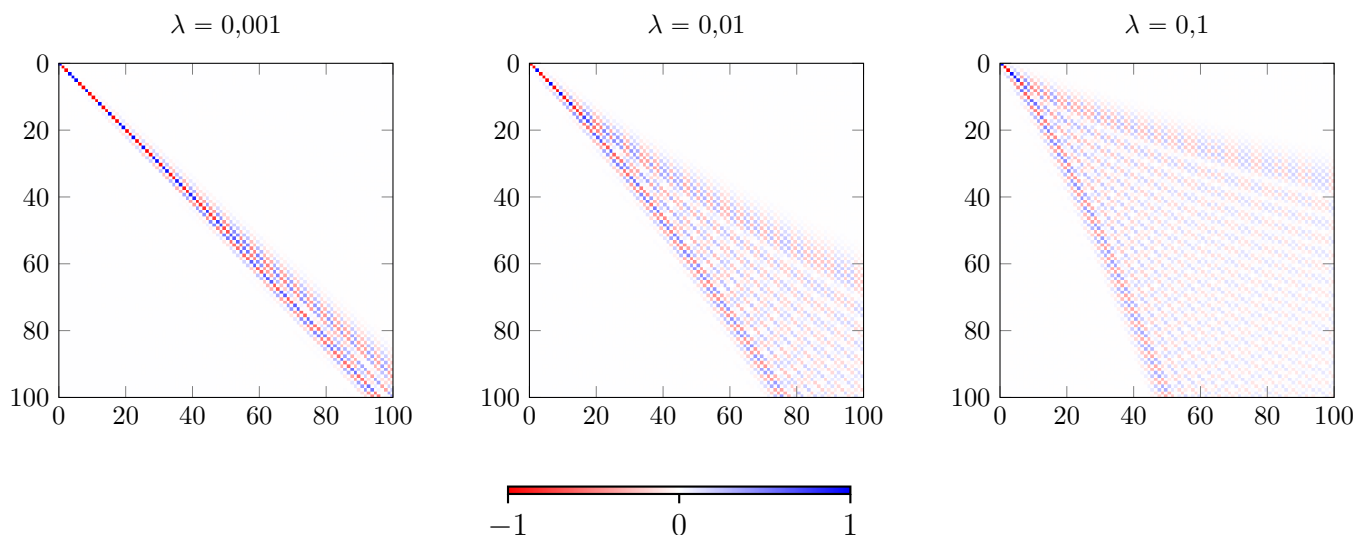
Za harmonski oscilator je matrika lastnih vektorjev kar identiteta, z anharmonskim popravkom pa se spremeni – pričakujemo zvezno spreminjanje in razširjanje diagonale (če lastne vektorje uredimo po naraščajočih lastnih vrednostih).



Graf 9: Lastne funkcije za E_0 pri različnih λ , $N = 1000$



Graf 10: Lastne funkcije za E_1 pri različnih λ , $N = 1000$



Slika 1: Prvih 100 lastnih vektorjev (100 komponent) za različne vrednosti λ , izračuni na $N = 1000$.

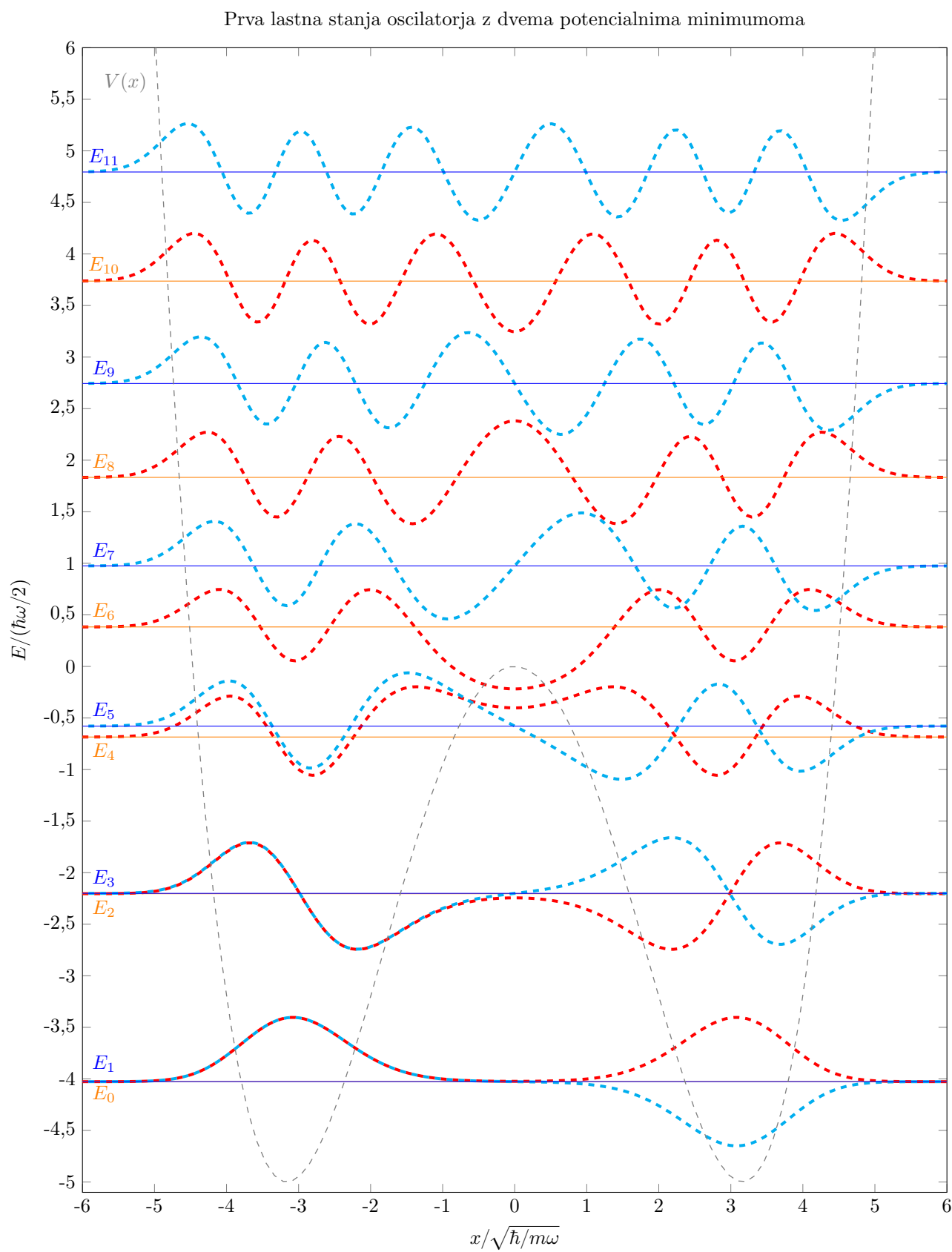
Kot pričakovano dobimo za večje vrednosti λ bolj razpršene lastne vektorje – širši spekter lastnih načinov harmonskega oscilatorja.

4.4 Potencial z dvema minimumoma

Za potencial z dvema minimumoma (graf 11) pričakovano dobimo na sredini pri lokalnem maksimumu potenciala $V(x)$ manjšo verjetnostno gostoto za stanja z nižjimi energijami. Ko energije stanj postanejo primerljive in večje od lokalnega maksimuma potenciala, njegov efekt začne bledeti – izginjati začnejo (skorajšnje) degeneracije nivojev (osnovno in prvo stanje se razlikujeta le za $\Delta E_{0,1} = 1,5 \cdot 10^{-5} \hbar \omega$, pri tem bi morali že preveriti ali ne gre morda le za numerično napako) in oblike lastnih funkcij okoli $x = 0$ postanejo bolj podobne tistim, ki opisujejo delec v potencialu brez lokalnega maksimuma, energijski nivoji pa postanejo bolj ekvidistančni.

5 Zaključek

V nalogi smo raziskali učinkovitost različnih metod za generiranje perturbacijskih matrik (q^4) ter različne metode diagonalizacije Hamiltonijanov (različne metode in različne knjižnice), ki so bili v našem primeru simetrične pasovne matrike. Za anharmonski oscilator smo poiskali lastne energije v odvisnosti od anharmoničnosti, določili njihovo natančnost oz. določili potrebno velikost Hamiltonijana, ki nam da želeno natančnost. Narisali smo tudi lastne vektorje v bazi lastnih stanj harmonskega oscilatorja, ki za večje anharmoničnosti postanejo bolj razpršeni okoli diagonale. Nekaj lastnih funkcij za različne anharmoničnosti smo tudi narisali. Pogledali smo si tudi primer potenciala z dvema minimumoma, kjer smo izračunali in narisali nekaj najnižjih lastnih stanj.



Graf 11: Prvih 12 lastnih energij in valovnih funkcij ($\psi_n(x)$ "naložen" na E_n) za potencial z dvema minimumoma, $N = 1000$.